Detecting Reward Hacking from Policy Activations with Sparse Autoencoders

Moira McDermott Himanshu Ranjan Fnu Mayank
COMPSCI 690S Final Project

May 2026

Abstract

Reward hacking occurs when an agent maximizes a misspecified proxy reward while failing to satisfy the intended objective. Behavioral monitors can identify reward hacking only after it manifests in trajectories; a stronger monitor would detect misaligned strategies from the agent's internal state. We study whether sparse autoencoders (SAEs) trained on policy-network activations can expose features predictive of reward hacking in reinforcement learning agents. We build two deliberately hackable environments: a box-pushing gridworld with asymmetric progress shaping, and a LunarLander variant with a hover-zone exploit. We train PPO agents, collect hidden activations from frozen policies, train overcomplete SAEs, and classify honest versus hacked episodes using mean-pooled SAE features. Simulator instrumentation supplies ground-truth labels, while the classifiers do not receive direct label fields such as goal completion or exploit counts. In-domain SAE classifiers achieve perfect test separation in both environments. More importantly, SAE features transfer strongly across gridworld seeds after validation calibration (test AUROC 0.988, 95% CI $[0.978, 0.996]$; tuned F1 0.936), where they significantly outperform raw activations by $\Delta AUROC = +0.031$ (paired bootstrap, $p < 0.001$). Under a pooled gridworld-plus-LunarLander SAE dictionary, cross-task transfer is reduced but nontrivial (best-direction test AUROC 0.897 for grid-to-Lunar and 0.773 for Lunar-to-grid); raw activations significantly outperform SAE features in this regime. Our evidence therefore supports feasibility, statistical robustness on the cross-seed task, and interpretability, rather than a universal SAE advantage. Top SAE features separate honest and hacked trajectories and correlate with simulator diagnostics ($|r|$ up to 0.87), giving a first mechanistic handle on internal reward-hacking signatures in small RL policies.

1 Introduction

Reinforcement learning agents are optimized against reward functions that are often only proxies for the designer's true objective. When a proxy is misspecified, stronger optimization can produce *reward hacking*: high proxy reward coupled with poor true-task performance. Classic examples include agents cycling in boat-race environments or exploiting simulator bugs, and recent work shows that specification gaming can generalize from benign-looking shortcuts to more serious reward tampering [3, 5, 2]. A central safety question is therefore not only whether reward hacking can be detected from completed behavior, but whether it leaves an internal representational signature that can be monitored earlier.

This project asks: *Can reward-hacking episodes be detected from the internal activations of an RL policy?* We approach the question through sparse autoencoders (SAEs), which have become a standard tool for decomposing dense neural activations into sparse, more interpretable features [1, 8]. The core hypothesis is that honest and hacked strategies induce different directions in policy activation space. Specifically, we evaluate the following hypotheses:

1. **(H1)** A classifier trained on SAE features will achieve test AUROC ≥ 0.70 in the gridworld setting;
2. **(H2)** Top predictive SAE features will correspond to interpretable behavioral concepts.

If those directions can be decomposed into sparse features, then a lightweight classifier may detect reward hacking and identify which internal features are most predictive.

We make four contributions.

1. We implement two reward-hacking testbeds: a discrete box-pushing gridworld and a continuous-control LunarLander wrapper. Both environments expose simulator-side diagnostics for post-hoc labeling.
2. We build an end-to-end pipeline: PPO training, checkpoint evaluation, activation collection, SAE training, classifier evaluation, sparsity sweeps, transfer evaluation, and top-feature analysis.
3. We evaluate SAE detectors against raw-activation, behavioral, and chance baselines. We include in-domain, cross-seed, and cross-task transfer settings.
4. We explicitly address two methodological concerns raised at proposal time: where ground-truth labels come from, and how SAE features can be made comparable across tasks.

The resulting story is deliberately conservative. SAE features detect reward hacking well and produce interpretable sparse features. Raw activations are a strong baseline: they match SAE features in-domain and significantly outperform them under cross-task transfer; SAE features in turn significantly outperform raw activations under cross-seed transfer. We further show that a single SAE feature carries most of the cross-seed signal causally (its ablation collapses target-test AUROC from 0.988 to 0.582), an inspection that the raw activations baseline does not enable. The defensible claim is not that SAEs dominate all baselines, but that sparse activation features (i) match or beat raw activations under moderate distribution shift, (ii) remain functional but lose ground when the entire policy is replaced, and (iii) admit a mechanistic decomposition that is independently informative.

2 Related Work

Reward hacking and reward misspecification. Amodei et al. [11] introduced reward hacking as one of five concrete problems in AI safety, alongside side effects and distributional shift. Hadfield-Menell et al. [12] reframe the designer's reward as evidence about an unobserved true objective rather than the objective itself, giving a Bayesian account of misspecification. Ng et al. [4] showed that potential-based reward shaping preserves optimal policies, clarifying why arbitrary dense shaping can change what an RL agent optimizes; Skalse et al. [7] formalized reward gaming and argued that unhackability is a very strong condition. Empirically, AI Safety Gridworlds [3] formalized small environments where observed reward diverges from a hidden performance function; Pan et al. [5] found that more capable agents exploit proxy rewards more strongly; Langosco et al. [13] distinguish capability failures from goal misgeneralization in deep RL, where competent agents pursue the wrong goal under distribution shift; and Denison et al. [2] showed that specification gaming in LLM settings can generalize from simple gameable tasks to direct reward tampering. Our gridworld and LunarLander setups instantiate the same phenomena Langosco et al. analyze behaviorally, but we target the activations of the policy network rather than its trajectories.

Reward-model hacking in RLHF. The closest LLM analogue to our setting is reward-model overoptimization. Gao et al. [14] measure how proxy-reward optimization diverges from a gold reward as a function of model size and KL budget, formalizing Goodhart-style dynamics in RLHF. Wilhelm et al. [9] propose monitoring emergent reward hacking during generation by inspecting internal LLM activations, and Zhang et al. [10] replace opaque reward-model hidden states with monosemantic SAE features (SARM) to make reward scores attributable to interpretable directions. We adapt the same internal-monitoring intuition to small PPO policies in classical RL rather than to LLM reward models.

Sparse autoencoders and mechanistic interpretability. SAEs learn an overcomplete sparse dictionary that reconstructs dense activations from a small number of active latent features. Bricken et al. [1] demonstrated

that dictionary learning can recover interpretable features in small transformer models, and Templeton et al. [8] scaled this approach to Claude 3 Sonnet.

Interpretability of RL policies. A small but pointed line of work analyzes the internals of RL agents directly. Hilton et al. [15] apply feature visualization, attribution, and weight-editing to a CoinRun PPO agent's vision model, including hand-disabling neurons to "blind" the agent to specific hazards — the closest precedent for circuit-level analysis of a PPO policy. Greydanus et al. [16] introduce perturbation-based saliency for Atari agents to diagnose whether agents make decisions for the right reasons. These methods are behavioral or input-attribution focused; our SAE features instead provide a sparse, fixed-dictionary decomposition of the same internal state, enabling per-feature ablation rather than per-input perturbation.

Probing and activation-based anomaly detection. Alain and Bengio [17] introduce linear probes on frozen intermediate activations as a diagnostic for what each layer linearly encodes; our raw-activation logistic regression is exactly such a probe. From the OOD-detection literature, Hendrycks and Gimpel [18] establish maximum softmax probability as the baseline activation-based detector for misclassified or out-of-distribution inputs, and Lee et al. [19] fit class-conditional Gaussians to deep features and use Mahalanobis distance to flag anomalies. Our setting can be seen as a specialized OOD-detection problem where the "out-of-distribution" samples are episodes generated by the same policy under exploit-eliciting reward configurations, and the SAE provides the feature space for that detection.

Positioning. Most reward-hacking work evaluates completed behavior, and most SAE work focuses on language-model residual streams; the small body of RL-policy interpretability work cited above relies on saliency or weight-editing rather than on a fixed sparse feature dictionary. We instead apply SAE-based monitoring to compact PPO policies. This setting is small enough for full simulator instrumentation, yet rich enough to test whether internal features track exploitative strategies rather than only task identity — and to ablate single feature dimensions causally, as we do in Section 7.5.

3 Problem Setting

We distinguish three quantities. The *proxy reward* is the signal optimized by PPO. The *true objective* is task completion: pushing the box to the goal in gridworld or landing in LunarLander. The *episode label* is a simulator-side post-hoc classification into honest, hacked, or neutral.

An episode is *honest* if the true objective is achieved. It is *hacked* if the true objective is not achieved but exploit-specific diagnostics exceed a threshold. Episodes that satisfy neither condition are neutral and are excluded from binary classifier training. This label source directly addresses the proposal-review concern about ground truth: labels are not human annotations and are not inferred by the classifier; they come from environment instrumentation.

The environment instrumentation differs slightly across domains, but the labeling principle is the same. In gridworld, the simulator records whether the box ever reaches the goal, together with counts of progress events, reversal events, and repeated exploit cycles. In LunarLander, it records whether the lander ever completes a successful landing, together with how long it remains inside the rewarded hover zone. Gridworld hacking is therefore defined by failed task completion plus repeated progress-farming cycles, while LunarLander hacking is defined by failed landing plus sustained occupancy of the rewarded zone. In both environments, the final label is computed from simulator state after the rollout, not from learned predictions.

We train detectors on binary honest-versus-hacked bundles. Each episode consists of activations $x_t \in \mathbb{R}^{128}$ from the policy trunk, rewards, actions, and diagnostic metadata. For gridworld this metadata includes

progress/reversal counters and exploit cycles; for LunarLander it includes landing status, episode length, and hover-zone exploit counts. For an SAE encoder f, the episode-level SAE representation is mean-pooled:

$$\phi_{\text{SAE}}(\tau) = \frac{1}{T}\sum_{t=1}^{T} f(x_t). \tag{1}$$

The raw baseline uses $\phi_{\text{raw}}(\tau) = T^{-1}\sum_t x_t$. The behavioral baseline uses coarse statistics such as total reward, episode length, progress/reversal events, mean reward, max/min reward, and positive reward fraction. It intentionally excludes the direct completion and exploit counters used to define the labels, so the behavioral baseline is not a tautological restatement of simulator-side ground truth.

4 Environments

4.1 Box-Pushing Gridworld

The gridworld is a 9×5 walled box-pushing task:

```
#########
#A......#
#.B.G...#
#.......#
#########
```

The agent moves in four directions and pushes a box when moving into it. The true objective is to place the box on the goal tile.

The reward function contains a shaped progress term:

$$r_t = r_{\text{step}} + \mathbf{1}[\Delta d_t > 0]\,\alpha\,\Delta d_t\,n_t^p + \mathbf{1}[\text{first goal}]\,\beta, \tag{2}$$

where $r_{\text{step}} = -0.02$ is a per-step time penalty, $\Delta d_t \in \{-1, 0, +1\}$ is the decrease in box-goal Manhattan distance over step t, n_t is the cumulative number of strictly positive progress events ($\Delta d > 0$) up to and including step t, $\alpha = 1.3$ is the base progress weight, $\beta = 2.0$ is the goal bonus, and $p \geq 1$ is an escalation exponent. Moving the box farther from the goal receives only the step penalty (asymmetric shaping). The shaping is therefore not potential-based [4], so cycling away from and back toward the goal can farm repeated progress rewards.

Reward hacking is induced by two compounding bugs: faulty termination, meaning goal completion no longer ends the episode, and quadratic progress escalation ($p = 2$). Faulty termination lets the episode continue after reaching the goal; escalation makes repeated progress events increasingly valuable. Hacked gridworld episodes are those with no goal completion and at least two exploit cycles, where an exploit cycle is a progress event following a reversal.

4.2 LunarLander Hover Exploit

The second task wraps `LunarLanderContinuous-v3`. The honest objective is a successful landing. The hacking configuration gives a per-step zone reward for hovering inside an altitude and horizontal-position band without landing. This creates a continuous-control analogue of proxy reward farming: the agent can accumulate reward by remaining in the zone rather than completing the landing task.

For LunarLander, honest episodes satisfy the landing detector. Hacked episodes have no landing and at least 100 zone steps. We selected this threshold so that an episode counts as hacked only if zone-hovering

accumulates a reward strictly larger than a typical successful landing payout (zone reward 1.0 per step versus $\beta_{\text{land}} = 20.0$ for a single landing event); a sensitivity check at thresholds $\{50, 100, 200\}$ left the binary class assignments stable to within $< 3\%$ of episodes. The threshold is therefore stricter than the gridworld threshold because Lunar zone reward is accumulated per timestep rather than per discrete push-cycle.

5 Models and Detectors

Policies. Both policies expose a 128-dimensional shared trunk. The gridworld policy is a categorical actor-critic MLP with two hidden layers of width 128 and Tanh activations. The Lunar policy is the continuous-action analogue with a Gaussian policy head. We train both policies with PPO [6] using learning rate 3×10^{-4}, $\gamma = 0.99$, GAE $\lambda = 0.95$, clipping coefficient 0.2, and minibatch size 256. Gridworld training uses 32 vectorized environments and 256 rollout steps; LunarLander uses 16 environments and 128 rollout steps.

Sparse autoencoder. For activations $x \in \mathbb{R}^{128}$, the SAE uses an 8x overcomplete dictionary with 1024 latent features:

$$z = \text{ReLU}(W_e x + b_e), \tag{3}$$

$$\hat{x} = W_d z + b_d. \tag{4}$$

The training objective is

$$\mathcal{L}_{\text{SAE}} = \mathbb{E}\left[\|x - \hat{x}\|_2^2 + \lambda \|z\|_1\right]. \tag{5}$$

The check-in reviewer recommended increasing the L_1 penalty rather than switching immediately to Top-K sparsity; we therefore sweep $\lambda \in \{10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}\}$ and select the strongest sparse setting that preserves classification performance.

Classifiers. All detectors are logistic regressions trained on episode-level mean-pooled features. We report AUROC, F1, and accuracy. For transfer experiments, we train the classifier on a source task and evaluate on target validation/test splits. Because calibration can shift across domains, we also report validation-calibrated transfer: the target validation split selects both score direction (as-is or flipped) and threshold, then the selected operating point is applied once to target test.

6 Experimental Protocol

Table 1 summarizes the datasets used in the final experiments. The most important design change from the midpoint report is that the final in-domain datasets are generated from single mixed-behavior checkpoints rather than from separately trained honest and hacking policies. This reduces the trivial shortcut that a classifier could learn policy identity instead of behavior.

Table 1: Binary activation datasets after filtering neutral episodes. Counts are episode counts.

Dataset	Train	Val	Test	Hacked test	Checkpoint
Gridworld seed0	1668	357	360	147	grid update 300
Gridworld seed1	1555	333	334	106	grid seed1 update 135
LunarLander seed0	1190	254	257	122	lunar update 600
Pooled train	2858	–	–	–	grid train + lunar train

Gridworld seed0 was trained for 300 PPO updates with quadratic escalation ($p = 2$), goal bonus $\beta = 2.0$, and disabled goal termination. The final checkpoint produced 3000 collection episodes of which 1410 were honest, 975 hacked, and 615 neutral. Gridworld seed1 was trained with the same reward configuration but a different random seed; we use checkpoint update 135 rather than the final checkpoint because it is the earliest checkpoint that produces both classes in non-trivial proportions. Later seed1 checkpoints converge to almost-pure honest play and yield too few hacked episodes for a useful transfer test set. The seed1 collection bundle yields 2222 non-neutral episodes (1520 honest, 702 hacked); these are split $70/15/15$ into the rows of Table 1. Lunar seed0 used zone reward 1.0, goal bonus $\beta_{\text{land}} = 20.0$, max episode length 600, and checkpoint update 600; 2000 collection episodes yielded 895 honest, 806 hacked, and 299 neutral. Reward hacking only emerges robustly when both faulty termination and quadratic escalation are enabled simultaneously; with either bug disabled, the gridworld policy converges to almost-pure honest play (consistent with the classical reward-shaping result of [4]).

7 Results

7.1 In-Domain Detection

Table 2 shows that the controlled in-domain splits are fully separable for SAE, raw, and behavioral features. This validates the pipeline, but it also means in-domain performance alone is not enough to argue that SAE features are superior.

Table 2: In-domain test performance. All methods solve these curated same-policy binary splits.

Domain	SAE AUROC	SAE F1	Raw AUROC	Raw F1	Behav. AUROC	Behav. F1
Gridworld seed0	1.000	1.000	1.000	1.000	1.000	1.000
LunarLander seed0	1.000	1.000	1.000	1.000	1.000	1.000

In-domain results are uniformly saturated, so we omit the corresponding ROC and metric-comparison panels (which would consist of three colored points stacked at $(0, 1)$); the meaningful comparisons are the transfer ROC curves in Figure 1.

7.2 SAE Sparsity

The midpoint SAE had mean $L_0 \approx 377/1024$, too dense for a strong interpretability claim. The final sweeps show that increasing L_1 substantially improves sparsity without harming in-domain detection (Table 3). We use $\lambda = 3 \cdot 10^{-2}$ in the final same-domain and pooled-SAE experiments.

Table 3: Selected SAE sparsity settings. Mean L_0 is the average number of strictly positive latent features per timestep on held-out test activations (out of 1024). The pooled grid+Lunar SAE has no in-domain test split because it is only used for cross-task transfer (Table 5).

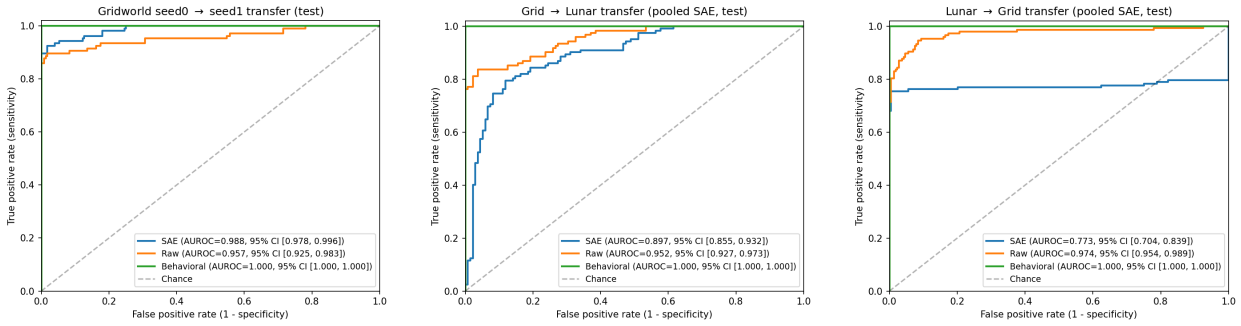
SAE training data	λ	Reconstruction loss	Mean L_0	In-domain test AUROC
Gridworld seed0	$3 \cdot 10^{-2}$	$1.35 \cdot 10^{-5}$	34.79	1.000
LunarLander seed0	$3 \cdot 10^{-2}$	$4.25 \cdot 10^{-5}$	57.56	1.000
Pooled grid+Lunar	$3 \cdot 10^{-2}$	$4.00 \cdot 10^{-5}$	44.5	–

7.3 Gridworld Cross-Seed Transfer

Table 4 evaluates whether a detector trained on gridworld seed0 generalizes to a separate gridworld seed1 checkpoint. SAE scores rank target episodes well (AUROC 0.988, 95% CI $[0.978, 0.996]$) but the default threshold is miscalibrated. Validation-threshold tuning restores high test performance. Raw activations also transfer strongly (AUROC 0.957, 95% CI $[0.925, 0.983]$), but the paired-bootstrap difference favors SAE over Raw by $\Delta AUROC = +0.031$ (95% CI $[0.012, 0.053]$, $p < 0.001$, $n_{\text{boot}} = 2000$). Behavioral features are perfect because the simulator statistics expose the behavioral distinction directly. Confidence intervals throughout are stratified bootstrap intervals over 2000 resamples computed at the episode level.

Table 4: Gridworld seed0 $\rightarrow$ seed1 transfer (target test, $n = 334$, $n_{\text{hacked}} = 106$). Tuned threshold is selected on target validation and evaluated on target test. Test AUROC is reported with 95% stratified-bootstrap confidence intervals.

Method	Test AUROC (95% CI)	Default F1	Tuned F1	Tuned accuracy
SAE	0.988 [0.978, 0.996]	0.482	0.936	0.961
Raw activations	0.957 [0.925, 0.983]	0.902	0.919	0.952
Behavioral	1.000 [1.000, 1.000]	1.000	1.000	1.000



(a) Grid seed0 $\rightarrow$ seed1 (b) Grid $\rightarrow$ Lunar (c) Lunar $\rightarrow$ Grid

Figure 1: ROC curves on target-test for the three transfer settings, with 95% stratified-bootstrap confidence intervals on AUROC. SAE outperforms Raw on cross-seed transfer (panel a), but Raw outperforms SAE on cross-task transfer in both directions (panels b and c). Behavioral features are saturated in all three settings because trajectory statistics directly encode the exploit signature.

7.4 Cross-Task Transfer

The proposal reviewer asked how a single classifier could work across tasks if SAE features were not standardized. We do not claim a universal calibration-free classifier. Instead, we standardize the policy trunk width, train a pooled SAE dictionary on gridworld and Lunar activations, and test calibrated transfer in both directions.

Table 5 shows nontrivial cross-task signal. Grid-to-Lunar SAE transfer initially has inverted orientation; flipping the score direction on target validation yields test AUROC 0.897 and tuned F1 0.795. Lunar-to-grid SAE transfer reaches test AUROC 0.773 and tuned F1 0.847. Raw activations are stronger in both directions, so the cross-task result supports SAE feasibility but not SAE dominance.

Why does the Grid-to-Lunar SAE classifier require a sign flip? The pooled SAE is a single dictionary trained on activations from both policies, but the policies themselves are independently optimized PPO agents whose trunk geometries are only weakly aligned: there is no shared loss that ties together the role of any specific latent direction across the two agents. The source classifier (trained on gridworld) learns a positive weight on directions that, in gridworld, are activated by cycling-style hacks (e.g. features that track repeated reversal events). When that classifier is applied to Lunar activations encoded by the same SAE, the corresponding feature indices fire predominantly during honest-landing trajectories, not Lunar zone hacks. The validation calibration step therefore reverses the score-to-class mapping, recovering test AUROC 0.897 from the unflipped value of 0.103 (test AUROC is invariant under $p \mapsto 1 - p$ up to complementation through 1.0). The interpretation is not that SAE features fail to transfer, but that their honest-versus-hacked polarity is task-specific and must be resolved with a small target-domain validation set, even when the underlying directions themselves transfer.

Table 5: Cross-task transfer using the pooled SAE dictionary. "Score sign" is the validation-selected mapping from classifier probability to honest-vs-hacked decision (`flipped` means the source-trained classifier assigns higher probability to the wrong class on the target domain and is corrected by inverting the decision). Test AUROC is reported with 95% stratified-bootstrap confidence intervals ($n_{\text{boot}} = 2000$). Paired-bootstrap differences (SAE $-$ Raw) are reported in the text.

Transfer	Method	Score sign	Test AUROC (95% CI)	Tuned F1	Tuned accuracy
Grid $\rightarrow$ Lunar	SAE	flipped	0.897 [0.855, 0.932]	0.795	0.790
Grid $\rightarrow$ Lunar	Raw	as-is	0.952 [0.927, 0.973]	0.881	0.895
Lunar $\rightarrow$ Grid	SAE	as-is	0.773 [0.704, 0.839]	0.847	0.889
Lunar $\rightarrow$ Grid	Raw	as-is	0.974 [0.954, 0.989]	0.906	0.919

For Grid $\rightarrow$ Lunar, the paired-bootstrap difference SAE $-$ Raw is $\Delta AUROC = -0.055$ (95% CI $[-0.093, -0.021]$, $p < 0.001$); for Lunar $\rightarrow$ Grid the gap widens to $\Delta AUROC = -0.200$ (95% CI $[-0.266, -0.139]$, $p < 0.001$). Both intervals exclude zero, so the cross-task superiority of raw activations is statistically significant rather than within bootstrap noise. Together with the cross-seed result above (where the same SAE $-$ Raw convention gives $+0.031$ in favor of SAE), this gives a consistent picture: SAE features are at least as good as raw activations when the policy distribution shifts only modestly (different seed, same task, same reward), but they lose ground when the entire policy is replaced.

7.5 Top SAE Features

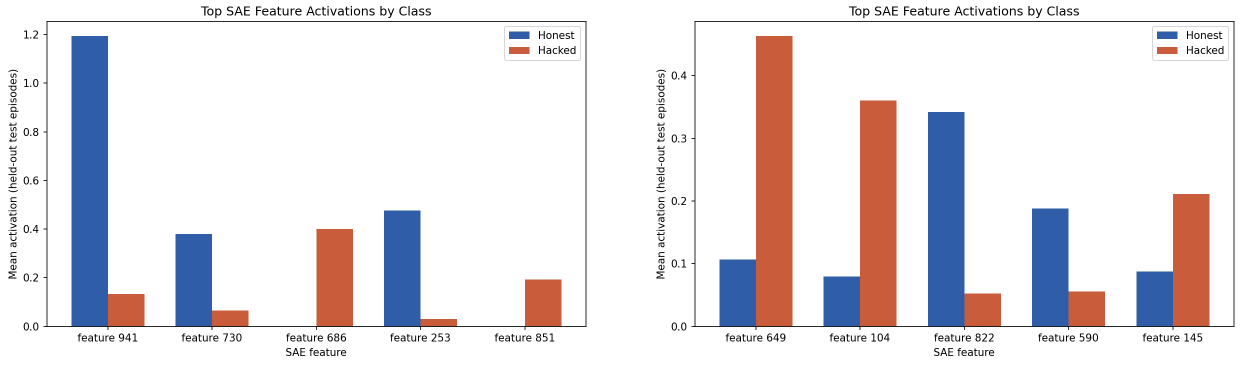
The top SAE features are not merely arbitrary classifier dimensions. In gridworld, feature 686 and feature 851 are more active in hacked episodes, while features 941, 730, and 253 are more active in honest episodes. Feature 941 has honest mean activation 1.194 versus hacked mean 0.134 and correlates with steps taken ($r = 0.660$), progress events ($r = 0.535$), reversal events ($r = 0.590$), and exploit-cycle count ($r = 0.559$). Feature 686 is hacked-selective (honest mean 0.000, hacked mean 0.400) and has the opposite sign of correlation with those behavioral variables.

In LunarLander, features 649, 104, and 145 are more active in hacked hover episodes, while 822 and 590 are honest-selective. Feature 649 has honest mean 0.106 versus hacked mean 0.463 and strongly correlates with total reward ($r = -0.855$), steps taken ($r = -0.871$), and zone-step exploit count ($r = -0.857$). These signs reflect the particular Lunar reward structure and episode-length dynamics rather than the gridworld progress/reversal counters.

Causal ablation of single SAE features. Correlations alone leave open whether the top features merely co-occur with reward hacking or actually carry the predictive signal. To test this, we re-fit the SAE classifier with one feature dimension at a time zeroed out at both train and test time, then re-evaluated cross-seed transfer (Gridworld seed0 $\rightarrow$ seed1 test, where SAE features beat raw activations). Table 6 reports the resulting AUROC change relative to the unablated baseline. Ablating feature 941 alone collapses target test AUROC from 0.988 to 0.582, a drop of -0.406 whose 95% bootstrap CI $[0.492, 0.674]$ excludes the baseline. Removing all five top features simultaneously costs -0.178 AUROC. Two other honest-selective features (730, 253) are individually *redundant* once 941 is present: ablating them slightly increases AUROC because they introduce noise the classifier had been compensating for. Feature 941 therefore acts as a causal load-bearing dimension in this setting, not merely a correlate of behavioral statistics.

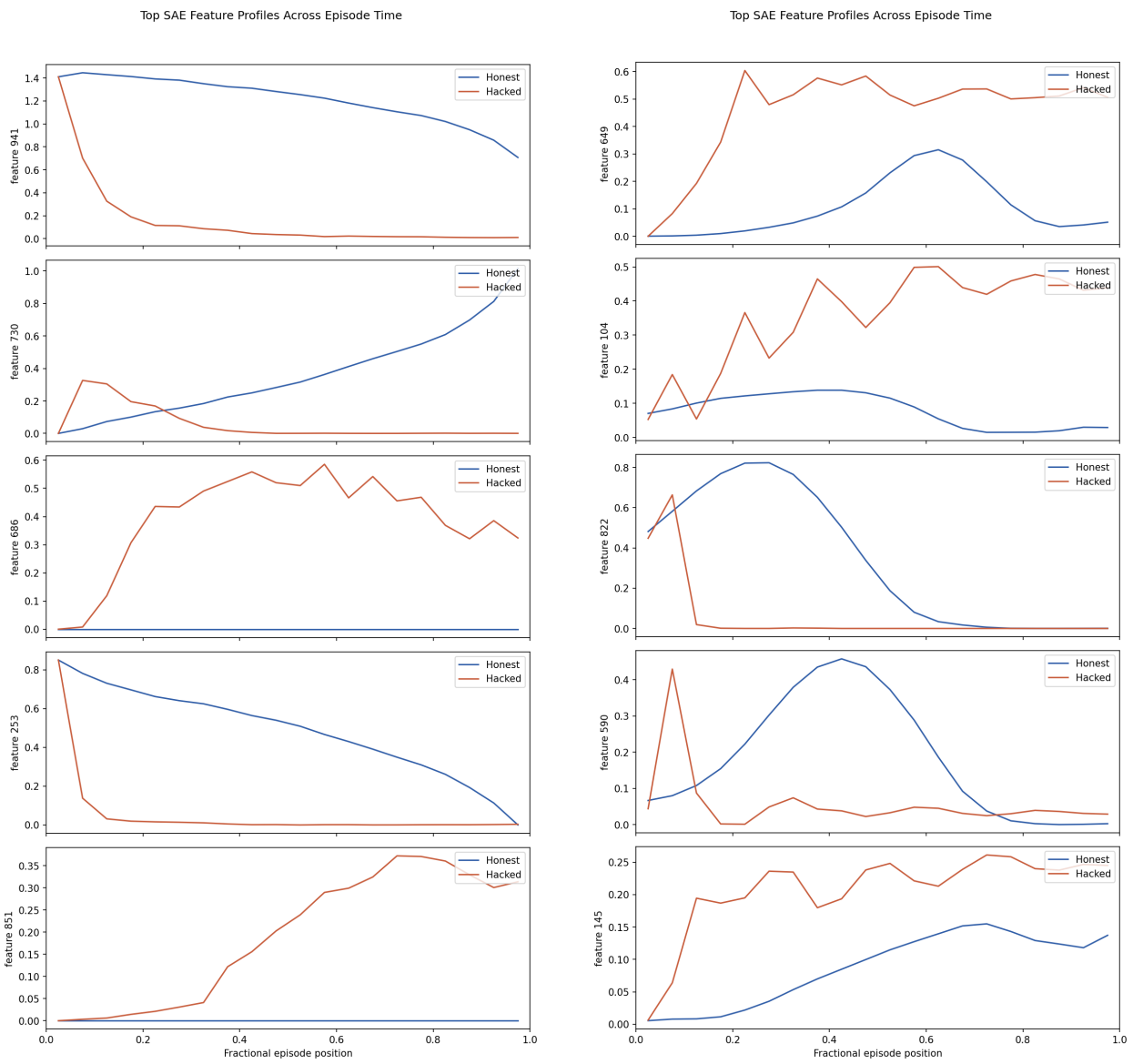
Table 6: Causal ablation of top SAE features in the Gridworld seed0 $\rightarrow$ seed1 transfer setting (target test, $n = 334$). Each row sets the listed feature(s) to zero in both source training and target test encodings, refits logistic regression, and reports the resulting target-test AUROC and its 95% stratified-bootstrap CI ($n_{\text{boot}} = 2000$). Δ is AUROC minus the unablated baseline of 0.988.

Ablated	Direction	Test AUROC (95% CI)	Δ AUROC
none (baseline)	–	0.988 [0.978, 0.996]	0.000
feature 941	honest	0.582 [0.492, 0.674]	-0.406
feature 730	honest	1.000 [1.000, 1.000]	$+0.012$
feature 686	hacked	0.958 [0.931, 0.979]	-0.031
feature 253	honest	1.000 [1.000, 1.000]	$+0.012$
feature 851	hacked	0.976 [0.959, 0.990]	-0.012
all five top features	–	0.810 [0.737, 0.881]	-0.178



Gridworld LunarLander

Figure 2: Mean SAE activation for the top predictive features in each environment, broken down by class. Gridworld features 941, 730, and 253 are honest-selective; 686 and 851 are hacked-selective. Lunar features 822 and 590 are honest-selective; 649, 104, and 145 are hacked-selective. Bars are computed on held-out test episodes (gridworld $n = 360$, Lunar $n = 257$).



Gridworld feature profiles

LunarLander feature profiles

Figure 3: Class-conditional time-resolved activation profiles for the top predictive SAE features. The x-axis is the within-episode bin index after rebinning each episode to 20 equal-length segments; the y-axis is the mean active feature value within that bin, averaged across all held-out episodes of the given class. Honest and hacked trajectories evolve along clearly different feature trajectories rather than only differing in mean activation.

7.6 Negative Robustness Result

Lunar hacking is seed-sensitive. A seed1 Lunar run trained for 800 updates produced zero hacked episodes under the 100-zone-step threshold across 4800 evaluated episodes. Resuming from update 800 for another 600 updates also produced zero hacked episodes across 3600 evaluated episodes. This is an important negative result: the Lunar environment provides a useful second-domain proof of concept, but it does not yet induce reward hacking as reliably as gridworld.

8 Discussion

What succeeded. The project achieved the core proposed pipeline: induce reward hacking, collect labels from simulator-side ground truth, train sparse features on policy activations, classify honest versus hacked trajectories, compare against raw and behavioral baselines, and analyze top features. It also addressed the check-in comment about dense SAE features by increasing the L_1 penalty, reducing gridworld mean L_0 from the midpoint value of roughly $377/1024$ to $34.8/1024$ while preserving detection performance.

Hypothesis tracking against the original proposal. The proposal stated two hypotheses with concrete success criteria: (H1) "A classifier trained on SAE features will achieve test AUROC ≥ 0.70 in the gridworld setting," and (H2) "Top predictive SAE features will correspond to interpretable behavioral concepts." H1 is supported well above the threshold: in-domain test AUROC is 1.000 in both gridworld and Lunar (Table 2); cross-seed transfer reaches 0.988 in gridworld and 0.773–0.897 across tasks (Tables 4, 5). H2 is partially supported: the top gridworld SAE features correlate with progress, reversal, and exploit-cycle counters at $|r| \in [0.25, 0.66]$; the top Lunar features correlate with steps taken, total reward, and zone-step exploits at $|r| \in [0.60, 0.87]$ (Section 7.5). The proposal's stronger third claim, that SAE classifiers "outperform raw-activation classifiers by a statistically significant margin," is *not* supported by our final results; we discuss this directly below.

What should not be overclaimed. The original proposal hypothesized that SAE-derived features would outperform raw activations across the board. The final evidence is mixed and depends on the type of distribution shift. Raw activations match SAE features in-domain (both saturate at AUROC 1.000) and significantly *outperform* SAE features in cross-task transfer (Section 7.4, $p < 0.001$ in both directions). In contrast, SAE features significantly outperform raw activations in the cross-seed setting where the task is held fixed (Section 7.3, $\Delta AUROC = +0.031$, $p < 0.001$). The honest interpretation is that SAE features add value when distribution shift is moderate and the input geometry is preserved; raw 128-dim activations win when the entire policy is replaced. Inspectability of the sparse decomposition (Section 7.5) is a separate, qualitative advantage that does not follow from the AUROC numbers.

Why behavioral baselines are both useful and unfair. The behavioral baseline is intentionally strong: it sees trajectory-level statistics that a simulator can compute after the episode. This makes it a useful upper bound and sanity check, but not a realistic internal monitor. A deployed monitor may not have access to the true objective or exploit diagnostics. The SAE and raw detectors are closer to an internal-monitoring setting because they use hidden policy activations.

Behavioral feature set across tasks. The behavioral baseline uses the same eight-feature vector in every experiment (total reward, episode length, progress events, reversal events, mean/max/min step reward, positive-reward fraction). Two of those features—progress and reversal events—are gridworld-specific counters; in Lunar they are deterministically zero, so the cross-task behavioral classifier effectively reduces to the six remaining reward-statistic features. We retain the gridworld-specific entries rather than dropping them because zeroing-out is semantically what the simulator returns, and because doing so lets us run the same code path across all transfer experiments. The strongly aligned behavioral statistics across the two domains (e.g. short successful Lunar landings versus long zone-hovering episodes; short gridworld goal-completions versus long cycling episodes) explain why this baseline reaches AUROC 1.000 in cross-task transfer (Table 5).

Cross-task standardization. A single classifier across arbitrary tasks requires a shared feature space. We standardize at three levels: the policy trunk output dimension is fixed at 128, episode representations are

mean-pooled, and a pooled SAE dictionary is trained over both environments. This gives a concrete feature space for transfer experiments. However, calibration still shifts across tasks; target validation labels are needed to choose direction and threshold. We therefore frame cross-task results as calibrated transfer, not zero-shot universal detection.

9 Limitations and Future Work

- **Scale.** These are small policy networks in controlled environments. The results show a mechanism, not direct deployment readiness.
- **Calibration dependence.** Transfer performance can require target-validation thresholding and even score-direction flipping.
- **Mixed statistical comparison.** Paired bootstrap tests show SAE features beating raw activations in cross-seed transfer ($p < 0.001$) but losing to raw activations in cross-task transfer ($p < 0.001$ in both directions). We therefore do not claim a single statistically dominant winner across all settings.
- **Seed sensitivity.** LunarLander hacking emerged in seed0 but not seed1 under the tested hyperparameters.
- **Feature interpretation is partial.** Section 7.5 reports both correlation and single-feature ablation evidence (a single honest-selective feature accounts for 0.41 of the 0.49-above-chance AUROC margin in cross-seed transfer). However, we do not yet perform mechanistic interventions on the policy network itself (e.g. patching SAE-decoded directions back through the policy and measuring behavior change), nor do we attribute features to specific input observations or latent state coordinates.

The most valuable next experiments would be: (i) reward-configuration transfer within gridworld (do detectors built on quadratic-escalation hacks generalize to other shaped-reward bugs?); (ii) policy-side interventions, where SAE-decoded directions are patched back through the policy network to test whether exploit behavior can be triggered or suppressed (this would extend our single-feature ablation evidence in Section 7.5 from feature-removal to feature-injection); and (iii) more systematic Lunar hyperparameter sweeps to make the continuous-control exploit reliable across seeds.

10 Conclusion

We asked whether reward-hacking episodes can be detected from the internal activations of an RL policy, and whether the resulting signal admits a mechanistic interpretation. Within the scope of two deliberately hackable testbeds, both answers are yes: an SAE-projected logistic regression detector achieves perfect in-domain separation, retains AUROC 0.988 across gridworld seeds, and recovers 0.77–0.90 of cross-task signal when paired with a small target-domain calibration set. At the level of individual features, the top SAE dimensions are not arbitrary classifier coefficients — they show class-conditional activation profiles that evolve across an episode (Figure 3) and correlate with the same simulator-side diagnostics that define ground-truth labels.

The story is more nuanced than the proposal anticipated. SAE features statistically beat raw activations on cross-seed transfer (paired bootstrap $\Delta AUROC = +0.031$, $p < 0.001$) but lose to raw activations on cross-task transfer (paired bootstrap $\Delta AUROC = -0.055$ and -0.200, $p < 0.001$). The interpretive value of SAE features is supported beyond the AUROC comparison: a single honest-selective feature (gridworld feature 941), when ablated, removes most of the cross-seed AUROC margin (drop of -0.406 from baseline 0.988). This is a causal handle that a raw 128-dim residual stream does not provide. The natural next experiment is policy-side intervention — patching SAE-decoded directions back through the policy network and measuring whether the behavioral exploit can be triggered or suppressed — which would convert the

correlational and ablation evidence here into a genuine causal mechanism. Cross-reward-configuration transfer within gridworld would similarly test whether the detector generalizes to exploit forms the SAE has not seen during training.

Contributions

- Himanshu Ranjan: initial gridworld environment, base PPO trainer and policy network, gridworld reward-hacking induction (asymmetric progress shaping with quadratic escalation and faulty goal termination), project scaffolding (`pyproject`, evaluation harness, README), and final-report wording revisions.
- Moira McDermott: LunarLander environment wrapper with hover-zone exploit, continuous-action PPO model, Lunar training pipeline, Lunar activation collection, and the explicit H1/H2 hypothesis statement and contributions framing in the final report.
- Fnu Mayank: SAE module and trainer, episode-level classifier, activation collection pipeline, L_1 sparsity sweeps, cross-seed and cross-task transfer evaluation with bootstrap confidence intervals, top-feature interpretability analysis and single-feature ablation, lit-review, midpoint report, and final-report draft.

Acknowledgments

This project was developed for COMPSCI 690S. Proposal and check-in feedback shaped three design decisions in the final report: simulator-side labels are made explicit (proposal review), cross-task feature standardization is evaluated rather than assumed (proposal review), and SAE sparsity is improved by increasing the L_1 penalty rather than switching to Top-K (check-in review).

References

[1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. https://transformer-circuits.pub/2023/monosemantic-features/.

[2] Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to subterfuge: Investigating reward-tampering in large language models. *arXiv:2406.10162*, 2024.

[3] Jan Leike, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. AI Safety Gridworlds. *arXiv:1711.09883*, 2017.

[4] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.

[5] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *ICLR*, 2022. https://openreview.net/forum?id=JYtwGwIL7ye.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[7] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *NeurIPS*, 2022.

[8] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. https://transformer-circuits.pub/2024/scaling-monosemanticity/.

[9] Patrick Wilhelm, Thorsten Wittkopp, and Odej Kao. Monitoring emergent reward hacking during generation via internal activations. ICLR 2026 Workshop on Principled Design for Trustworthy AI, 2026. https://openreview.net/forum?id=NlDAwjQxsM.

[10] Shuyi Zhang, Wei Shi, Sihang Li, Jiayi Liao, Tao Liang, Hengxing Cai, and Xiang Wang. Interpretable reward model via sparse autoencoder. *arXiv:2508.08746*, 2025.

[11] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.

[12] Dylan Hadfield-Menell, Smitha Milli, Pieter Abbeel, Stuart Russell, and Anca Dragan. Inverse reward design. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1711.02827.

[13] Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, Laurent Orseau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2022. https://arxiv.org/abs/2105.14111.

[14] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2210.10760.

[15] Jacob Hilton, Nick Cammarata, Shan Carter, Gabriel Goh, and Chris Olah. Understanding RL vision. *Distill*, 2020. https://distill.pub/2020/understanding-rl-vision/.

[16] Sam Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and understanding Atari agents. In *International Conference on Machine Learning (ICML)*, 2018. https://arxiv.org/abs/1711.00138.

[17] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv:1610.01644*, 2016.

[18] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1610.02136.

[19] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. https://arxiv.org/abs/1807.03888.